

2.4.2 Searching Algorithms: Binary Search

بائنری سرچ (Binary Search) ایک تیز رفتار طریقہ ہے جو ہر قدم پر تلاش کے حصے کو آدھا کر دیتا ہے۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 09 of 11

★ Golden

2.4.2 Searching Algorithms: Binary Search

★ Golden

LEARN IT

Binary Search is a highly efficient search algorithm that works by repeatedly dividing the search interval in half. **Crucial limitation: it only works on a SORTED list.**

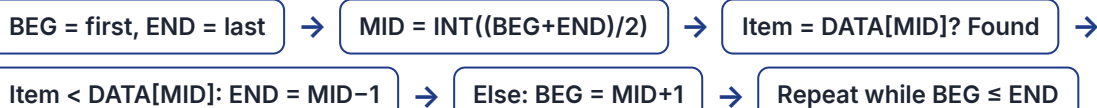
Three variables mark the part of the list still being searched:

- **BEG** (low) — first position of the current part.
- **END** (high) — last position of the current part.
- **MID** = $\text{INT}((\text{BEG} + \text{END}) / 2)$ — the middle position. INT means "drop the decimal part".

At each step compare the item with $\text{DATA}[\text{MID}]$:

- Equal $\rightarrow$ found, stop.
- Item $< \text{DATA}[\text{MID}] \rightarrow$ the item can only be in the **left** half: set $\text{END} = \text{MID} - 1$.
- Item $> \text{DATA}[\text{MID}] \rightarrow$ the item can only be in the **right** half: set $\text{BEG} = \text{MID} + 1$.

If **BEG** becomes greater than **END**, the part is empty and the item is **not in the list**.



Algorithm (textbook summary)

Pseudocode

```

Step 1: Start
Step 2: DATA := {1,2,3,4,5,6,7,8,9}
        (Sorted)
Step 3: BEG = 1, END = 9
Step 4: MID = INT((BEG+END)/2)
Step 5: Item = 7
Step 6: LOC = 0
Step 7: Repeat while BEG ≤ END AND
        DATA[MID] ≠ Item
Step 8:   If Item < DATA[MID]:
            END = MID - 1
        Else:
            BEG = MID + 1
Step 9:   MID = INT((BEG+END)/2)
Step 10: [End loop]
Step 11: If DATA[MID] = Item:
            LOC = MID
        Else:
            LOC = 0
Step 12: Print result based on LOC
Step 13: End

```

```

Start
DATA = {1,2,3,4,5,6,7,8,9}
BEG = 1, END = 9
MID = INT((BEG+END)/2)
Item = 7
LOC = NULL
While BEG ≤ END AND DATA[MID] ≠ Item
    If Item < DATA[MID] then
        END = MID - 1
    Else
        BEG = MID + 1
    End If
    MID = INT((BEG+END)/2)
End While
If DATA[MID] = Item then
    LOC = MID
Else
    LOC = NULL
End If
If LOC ≠ NULL then
    Print DATA[MID]
Else
    Print "Not Found"
End If
End

```

COMMON MISTAKE

The slide's pseudocode writes the loop condition as `BEG ≤ END OR DATA[MID] ≠ Item`. That is a slip: with OR the loop would never stop once the item is found while `BEG ≤ END`. The textbook algorithm (Step 7) correctly uses **AND** — keep going only while the part is not empty *and* the item is not yet found. Likewise, the slide sets `LOC = NULL` but then tests "`LOC >= 0`"; the check should simply be "`LOC is not NULL`" (or, with `LOC = 0` meaning not found, "`LOC > 0`"). The pseudocode above shows the corrected form.

WORKED EXAMPLE

Worked example 1 (textbook algorithm). DATA = 1, 2, 3, 4, 5, 6, 7, 8, 9; Item = 7. Positions 1 to 9.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	9	$\text{INT}(10/2) = 5$	5	$7 > 5$	BEG = 6 (go right)
2	6	9	$\text{INT}(15/2) = 7$	7	$7 = 7$	Found, LOC = 7

Only **2 comparisons**. Linear Search would need 7.

WORKED EXAMPLE

Worked example 2 (slide example). Sorted list: 1, 2, 3, 4, 5, 6, 7, 8. Searching item: 3.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	8	$\text{INT}(9/2) = 4$	4	$3 < 4$	END = 3 (go left)
2	1	3	$\text{INT}(4/2) = 2$	2	$3 > 2$	BEG = 3 (go right)
3	3	3	$\text{INT}(6/2) = 3$	3	$3 = 3$	FOUND at position 3

Picture the list shrinking (the highlighted cell is MID):

Step	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	1	2	3					
3			3					

The slide describes the same steps using the size of each sub-list: Step 1 MID = $8/2 = 4$ (value 4), $3 < 4$, go left; Step 2 sub-list [1, 2, 3], middle = $3/2 = 1.5 \rightarrow$ 2nd item (value 2), $3 > 2$, go right; Step 3 sub-list [3], middle = 1st item (value 3), FOUND. Both ways land on the same values $4 \rightarrow 2 \rightarrow 3$.

WORKED EXAMPLE

Binary Search in Python (index starts at 0, so positions 1–8 become indexes 0–7):

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
item = 3
beg = 0
end = len(data) - 1
loc = -1
step = 0
while beg ≤ end:
    mid = (beg + end) // 2
    step = step + 1
    print("Step", step, ": beg =", beg, "mid =", mid, "end =", end, "value =",
data[mid])
    if data[mid] == item:
        loc = mid
        break
    elif item < data[mid]:
        end = mid - 1
    else:
        beg = mid + 1
if loc ≥ 0:
    print("Found", item, "at index", loc)
else:
    print("Not Found")
```

OUTPUT

```
Step 1 : beg = 0 mid = 3 end = 7 value = 4
Step 2 : beg = 0 mid = 1 end = 2 value = 2
Step 3 : beg = 2 mid = 2 end = 2 value = 3
Found 3 at index 2
```

// is integer division, the same as INT(). Index 2 is position 3. Here `loc = -1` means "not found", so the test `loc ≥ 0` is correct.

Why is it so fast?

Every comparison throws away half of the remaining list. $1,000,000 \rightarrow 500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$ takes about 20 halvings, because $2^{20} = 1,048,576$. So Binary Search finds an item among 1 million elements in at most about **20 steps**, while Linear Search may need 1,000,000.

Advantages and disadvantages

Advantages	Disadvantages
Extremely fast for "Big Data": finds an item in 1M elements in just ~20 steps.	The list must be sorted first; it fails on scrambled data.

Massive time savings for static lists that are searched repeatedly.	Requires random access to elements (works on arrays, not easily on linked lists).
	More complex code, prone to logic errors (off-by-one).

REMEMBER IT

Binary Search is how you find a word in a dictionary: open the middle, decide "before or after?", and throw away half the book each time. **"Sorted first, then split."**

COMMON MISTAKE

Common errors: (1) using Binary Search on an unsorted list — the answer will be wrong; (2) setting $END = MID$ instead of $MID - 1$ (or $BEG = MID$ instead of $MID + 1$), which can cause an infinite loop — this is the "off-by-one" error; (3) forgetting INT , so MID becomes a decimal like 4.5.

اردو میں سمجھیں

بائنری سرچ (Binary Search) ایک تیز رفتار طریقہ ہے جو ہر قدم پر تلاش کے حصے کو آدھا کر دیتا ہے۔ یہ صرف ترتیب شدہ (Sorted) فہرست پر کام کرتا ہے۔ درمیانی عنصر (MID) سے موازنہ کیا جاتا ہے: اگر مطلوبہ چیز چھوٹی ہو تو بائیں آدھے حصے میں، اور بڑی ہو تو دائیں آدھے حصے میں تلاش جاری رہتی ہے۔ دس لاکھ عناصر میں بھی تقریباً بیس قدموں میں نتیجہ مل جاتا ہے۔

EXAM TIP

You will usually be given a sorted list and an item and asked to "search using Binary Search, showing all steps". Always draw a table with **BEG, END, MID, DATA[MID], comparison, action**. Show the INT calculation for MID . If the item is absent, continue until $BEG > END$ and write "Not Found". Also learn: "Why must the list be sorted?" and "Compare Linear and Binary Search".

CHECK YOURSELF

1. What is the crucial condition for Binary Search to work?
2. $BEG = 6$, $END = 9$. What is MID ?
3. In 1, 2, 3, 4, 5, 6, 7, 8 (positions 1–8), list the MID values visited when searching for 8.

Answers: 1. The list must be sorted. 2. $INT(15/2) = 7$. 3. $MID = 4$ (value 4, go right, $BEG = 5$) → $MID = INT(13/2) = 6$ (value 6, go right, $BEG = 7$) → $MID = INT(15/2) = 7$ (value 7, go right, $BEG = 8$) → $MID = 8$ (value 8, found). Four comparisons.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Binary search requires:

- a. A sorted list
- b. An unsorted list
- c. A graph
- d. A pie chart

MCQ 2. Each step discards:

- a. About half the remaining items
- b. One item only
- c. The whole list
- d. Only even numbers

MCQ 3. Complexity is:

- a. $O(\log n)$
- b. $O(n)$
- c. $O(n^2)$
- d. $O(1)$ always

MCQ 4. If mid value is too big you go:

- a. Left half
- b. Right half
- c. To a stack
- d. To OSI layer 7

MCQ 5. On sorted [1..8] searching 3, first mid is:

- a. 4 (value 4), then go left
- b. 1
- c. 8
- d. 7

MCQ 6. Binary search on unsorted data is:

- a. Wrong / may miss
- b. Always faster
- c. A sort
- d. A gate

Extra short questions

1. Define binary search and its precondition.
2. Why $O(\log n)$?
3. What if the list is unsorted?
4. Trace 3 in [1,2,3,4,5,6,7,8].

Extra long question

1. Write binary-search pseudocode with low, high, mid. Trace two targets (present and absent) on an 8-item sorted list. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Repeatedly test the middle of a sorted list; go left or right. List must already be sorted.
2. $n \rightarrow n/2 \rightarrow n/4 \dots$ until 1. About $\log_2 n$ steps.
3. Sort first (extra cost) or use linear search.
4. mid 4; $3 < 4$ left [1,2,3,4]; mid 2; $3 > 2$ right [3,4]; mid 3 found.

Long

1. WHILE $low \leq high$: $mid = (low + high) // 2$; equal return; less $\rightarrow high = mid - 1$; greater $\rightarrow low = mid + 1$. Absent: $low > high \rightarrow$ not found. Show the shrinking intervals.

Next lecture: 2.5 Evaluation Criteria of Algorithm